

Pretrained INT8 Transformer Layer Validation on an Agilex 5 Systolic Array

DistilBERT first attention query projection

Abstract

A pretrained transformer projection was executed on a 16 x 16 signed INT8 systolic-array coprocessor controlled by Linux on the Agilex 5 HPS. The evaluation used pretrained DistilBERT weights and activations captured from real tokenized text. Five projection shapes were tested with $K=N=768$ and M ranging from 1 to 512. [5-7]

All 150 measured runs, 15 warmups, and the smoke test passed exact integer verification. The exported floating-point layer outputs provided a separate measure of quantization approximation. Relative L2 error ranged from 2.37% to 2.78%, with cosine similarity above 0.9996. [6]

For tile-aligned shapes, useful wall throughput remained approximately 0.133-0.137 GOPS. The one-row case achieved 6.25% padding efficiency and took nearly as long as the 16-row case. Completion waits and operand packing dominated the measured HPS driver latency. The scalar CPU reference was faster end to end for every tested shape. [6]

Evaluation	Result
Arithmetic	Signed INT8 operands; INT32 accumulation
Correctness	Zero mismatches in 166 board invocations
Largest workload	512 x 768 activations multiplied by 768 x 768 weights
Largest median wall time	4.396 seconds; useful throughput 0.1374 GOPS
Validated scope	One pretrained query projection; FPGA GEMM plus host output comparison

This evaluation closes the project with a real pretrained workload in addition to synthetic GEMM validation. It establishes arithmetic correctness and measures the complete driver cost; it does not establish full-model task accuracy or an application speedup.

1. What the query projection does

DistilBERT is a compact transformer encoder obtained through knowledge distillation from BERT. Its pretrained representations can be adapted to text classification and question answering. [2]

Within self-attention, each input token is mapped to learned query, key, and value vectors. A query is compared with the keys of other tokens to produce attention scores. After normalization, those scores weight the value vectors. The resulting representation incorporates information from the surrounding sequence. [1]

The query is an internal learned vector, not a question typed by a person. The evaluated projection computes $Q = X W_Q^T + b_Q$, producing 768 features for each input row. DistilBERT divides these features among 12 attention heads with 64 features per head. [5]

Attention stage	Operation	Evaluated here?
Query projection	$X W_Q^T + \text{bias}$	INT8 GEMM on FPGA; scale and bias on host
Key and value projections	Generate keys and values	No
Attention scores	$Q K_{\text{att}}^T / \text{sqrt}(64)$	No
Attention output	$\text{softmax}(\text{scores}) V$	No

K_{att} denotes the attention key matrix. In the benchmark, $K=768$ instead denotes the GEMM reduction dimension. These are different uses of the letter K.

Real-world examples

Review sentiment: a fine-tuned DistilBERT classifier can categorize a review as positive or negative. In an illustrative review such as "The battery lasts all day, but the screen is disappointing," contextual attention helps form a representation of the contrasting clauses. An official SST-2 fine-tuned checkpoint demonstrates this task. [3]

Question answering: a fine-tuned model can locate an answer span in supplied text. For example, given "The warranty lasts two years" and "How long is the warranty?", the application returns the relevant span. An official SQuAD checkpoint provides this type of extractive question answering. [4]

These examples explain where query projections are used. They are illustrative applications, not tasks executed by this FPGA experiment. The FPGA evaluation used the base pretrained checkpoint and stopped at the first query projection.

2. Workload, quantization, and validation

The source checkpoint was **distilbert/distilbert-base-uncased**, fixed at revision 12040accade4e8a0f71eabdb258fecc2e7e948be. The selected module was `transformer.layer.0.attention.q_lin`. The experiment captured its input and floating output after embedding and normalization, using one real tokenized text sequence. [5,7]

M=1,16,64,128,512 selects the first M rows of that same 512-token capture; the one-row case is the CLS row. The cases are not five independently tokenized sentences. All share the pretrained 768 x 768 weight matrix. The PyTorch weight layout was transposed to the row-major GEMM operand B[K,N]. [7]

Symmetric quantization

Activations use one scale per case and weights use one scale per output channel. Zero points are zero. Nonzero scales equal the maximum absolute value divided by 127; a zero-valued tensor or channel uses scale 1. Values are rounded to nearest with ties to even and clipped to [-127,127]. [7]

Quantity	Definition
Integer GEMM	$C_{int} = A_{int8} B_{int8}$
Decoded layer output	$C_{int}[r,c] \times s_A \times s_B[c] + bias[c]$
Floating reference	Original PyTorch query-projection output, including bias

The accelerator performs signed INT8 multiplication with INT32 accumulation. The host converts scales and adds the pretrained bias for the floating comparison. At K=768, the largest possible signed INT8 dot product remains within INT32 range. [7]

Validation and measurement

An INT64 NumPy multiplication generated each exact integer reference. The C++ scalar reference recomputed every exported result before hardware access. Each FPGA invocation then compared every returned integer element with the reference. Package checksums associated each board case with its exported data. [6,7]

Each shape used three excluded warmups and 30 timed runs, with fixed operands and busy polling. The 16-row smoke test used one measured invocation and no warmups. All summaries were reconciled with the raw measured rows, including median, nearest-rank p95, maximum, geometry, throughput, and timing-stage medians. [6]

Wall time surrounds the entire `run_tiled_gemm` driver call: reset, packing, DMA submission, waits, and readback. Package loading, CPU reference, verification, scale conversion, bias addition, console output, and CSV writes are excluded. Thus the measured quantity is HPS-to-FPGA GEMM latency, not complete model or floating-to-floating application latency. [6,7]

3. Measured latency and throughput

M	Median wall (ms)	p95 (ms)	Maximum (ms)	Useful GOPS	Scalar CPU median (ms)
1	136.07	136.10	136.24	0.0087	3.07
16	142.06	142.17	142.21	0.1329	108.68
64	567.74	567.84	567.89	0.1330	432.71
128	1,135.46	1,135.64	1,135.72	0.1330	861.85
512	4,395.53	4,395.95	4,396.00	0.1374	1,547.61

Table 1. Thirty measured runs per shape. K=N=768. Useful GOPS counts one multiply and one add: 2MKN / wall time. CPU values are measured scalar INT8-to-INT32 reference timings, not optimized BLAS or PyTorch inference. [6]

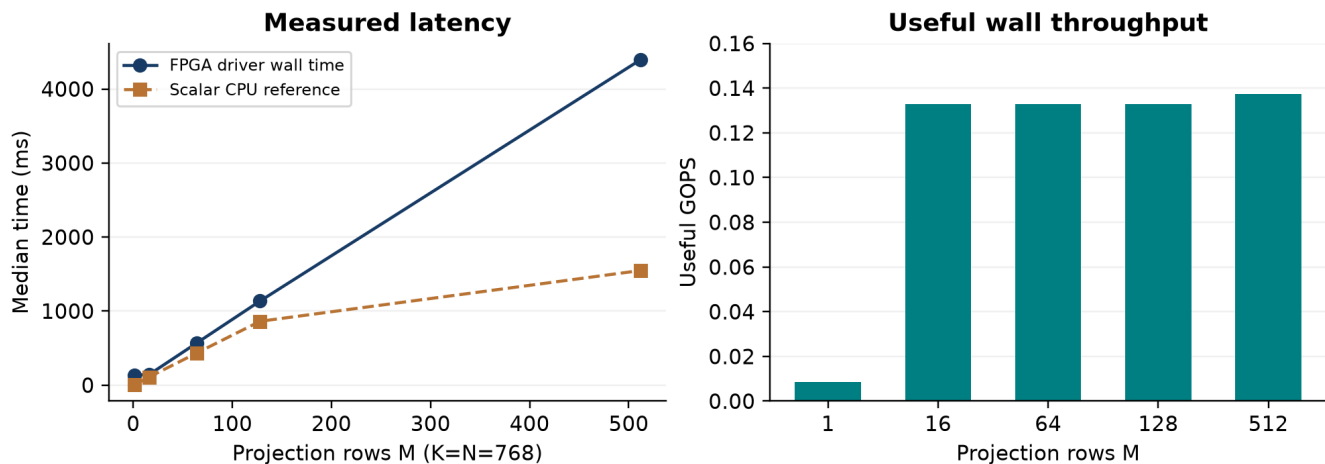


Figure 1. Median latency and useful wall throughput. All values are derived from the recorded board CSVs. The CPU curve is the scalar reference comparator. [6]

Scaling behavior

From M=16 to M=128, latency rises almost proportionally with output-tile count: approximately 142 ms, 568 ms, and 1,135 ms. M=512 takes 4,396 ms and sustains slightly higher throughput. No sharp throughput drop appears among these sampled shapes. [6]

The CPU reference is faster in every case. FPGA wall latency is approximately 1.31 times the CPU reference at M=16 and 2.84 times at M=512. The CPU timing does not scale uniformly between cases; these measurements alone do not establish the cause. [6]

The M=1 case processes 16 times fewer useful operations than M=16 but executes the same tile steps. Its latency is only about 4.2% lower. This directly exposes the cost of zero padding in the current host execution path. [6,7]

4. Tile geometry and host timing

M	Output tiles	K-accumulation tile steps	Useful operations (M)	Padding efficiency
1	48	2,304	1.180	6.25%
16	48	2,304	18.874	100.00%
64	192	9,216	75.497	100.00%
128	384	18,432	150.995	100.00%
512	1,536	73,728	603.980	100.00%

Table 2. Output tiles = $\text{ceil}(M/16) \times \text{ceil}(N/16)$. Each output tile accumulates 48 K steps. Padding efficiency = useful operations / (tile steps $\times$ 8,192). It is a geometry ratio, not measured hardware utilization. [6,7]

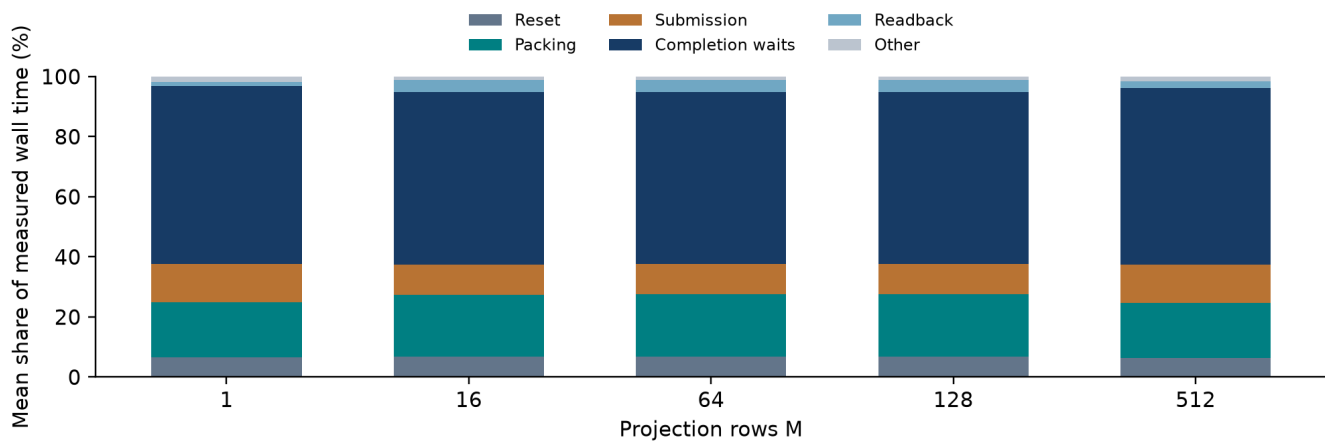


Figure 2. Mean stage fractions across 30 measured runs. Each fraction is computed against its own run wall time before averaging, so the displayed stack sums to 100%. [6]

Completion waits account for approximately 57-59% of measured wall time, while packing accounts for approximately 18-21%. Reset, submission, readback, and the remaining driver work make up the balance. These measurements identify where host time is spent; they do not isolate the individual hardware or software causes inside each wait. [6]

The current driver sequentially prepares and submits padded 16 x 16 tiles, accumulates through K, and drains results. The experiment exercises tiling and accumulation, but does not establish overlapped host load, compute, and drain. [7]

Hardware cycle count versus wall time

The hardware counter reports 145,248 cycles for M=16 and 4,647,936 cycles for M=512. At the configured 100 MHz assumption, these correspond to 1.452 ms and 46.479 ms, respectively: about 1% of the corresponding wall times. The counter covers PE reset, compute, and flush; it is not a timer for the complete DMA and host execution path. [6,7]

5. Quantization approximation

M	RMSE	Maximum absolute error	Relative L2 error (%)	Cosine similarity
1	0.02179	0.07089	2.784	0.9996159
16	0.02409	0.13608	2.563	0.9996716
64	0.02409	0.15179	2.441	0.9997024
128	0.02435	0.13951	2.374	0.9997184
512	0.02674	0.13587	2.390	0.9997145

Table 3. Decoded INT32 projection plus pretrained bias compared with the original floating-point projection output. Every FPGA integer output matched the verified integer reference. [6,7]

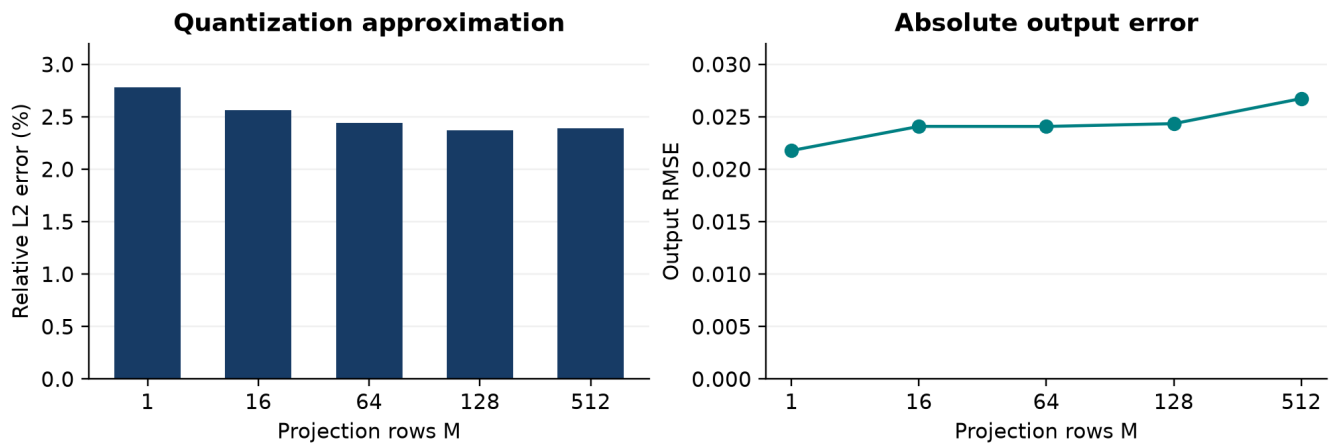


Figure 3. Relative and absolute approximation error for the five captured prefixes. These are layer-output metrics, not model accuracy measurements. [6]

Metric definitions

For decoded output Y and original floating output F , with L output elements: $RMSE = \sqrt{\sum((Y-F)^2)/L}$; maximum absolute error = $\max(\text{abs}(Y-F))$; relative L2 error = $\|Y-F\|_2 / \|F\|_2$; cosine similarity = $\text{dot}(Y,F) / (\|Y\|_2 \|F\|_2)$. Metrics use all valid outputs. [7]

Interpretation

Relative L2 error remains between 2.37% and 2.78%, and cosine similarity remains above 0.9996. Integer arithmetic correctness is exact; the nonzero floating difference is the expected approximation introduced by the selected quantization scheme. [6,7]

Each prefix has its own activation scale, so the M cases do not share exactly the same activation quantization. The floating comparison is calculated from the verified integer reference, which equals every accepted FPGA output, followed by host scale conversion and bias addition. Those operations are outside the wall timer. [7]

No downstream classifier, question-answering head, full attention block, or complete DistilBERT model was evaluated. Therefore these results do not quantify task-accuracy changes caused by quantization.

6. Project conclusion

The accelerator successfully executed a real pretrained DistilBERT query-projection workload using signed INT8 operands, INT32 accumulation, padded tiling, and K-block accumulation. All 166 board invocations passed, including 150 measured runs, 15 warmups, and the preliminary smoke test. The result extends the project beyond synthetic matrix inputs while preserving exact integer verification. [6,7]

The sampled tile-aligned shapes sustain approximately constant useful wall throughput, while the one-row case exposes the padding penalty. Quantization error is measured independently against the original floating layer. Complete driver latency remains above the scalar CPU reference in all five cases, and the stage measurements show that completion waits and packing dominate. [6]

The completed evaluation establishes functional integration and measured scaling for this projection. Its claims are limited to the captured input sequence and the tested layer; complete model inference, optimized-CPU speedup, and downstream task accuracy are outside the experiment.

References

- [1] A. Vaswani et al., Attention Is All You Need, 2017. Scaled dot-product and multi-head attention. [Source](#).
- [2] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter, 2019. [Source](#).
- [3] DistilBERT SST-2 English model card. Official fine-tuned sentiment-classification example. [Source](#).
- [4] DistilBERT distilled SQuAD model card. Official extractive question-answering example. [Source](#).
- [5] DistilBERT checkpoint and Transformers documentation. Base model, hidden dimension, and attention configuration. [Source](#).
- [5, continued] Hugging Face, DistilBERT implementation and configuration documentation. [Documentation](#). External sources accessed October 4, 2026.
- [6] Recorded board measurements: `distilbert_layer_01_m{1,16,64,128,512}_{runs,summary,quantization}.csv` and corresponding manifests; `distilbert_smoke_m16` files. Tables and figures are calculated from these measurements. Source files are retained in `docs/distilbert_layer_report_assets/source_data`.
- [7] Project implementation and export record: `test/gemm_test.cpp`, `test/gemm_layer.hpp`, `test/gemm_suite.hpp`, `scripts/export_distilbert_layer.py`, `scripts/layer_package.py`, and `model_data/distilbert_query/export_manifest.json`. These define packing, capture, quantization, reference checks, and timing boundaries.